PAC-MAN JAVA GAME

Object-Oriented Programming

Universita degli Studi di Messina
Professor: Salvatore Distefano
Mehdi Talebikhatir 558948

Contents

Abstract

This report explains how the Java Pac-Man project applies the main principles of Object-Oriented Programming. The project is built around reusable game objects, state objects, tile objects, input

handlers, file utilities, and rendering helpers. The report focuses on the concrete OOP concepts used in the code: inheritance, encapsulation, information hiding, abstraction, polymorphism, interface implementation, composition, subtyping, exception handling, customization, and the Open/Closed Principle.

Project Introduction

The project is a desktop Pac-Man-style arcade game implemented in Java for an Object-Oriented Programming course. The player controls Pac-Man inside a maze, collects dots, avoids ghosts, moves between levels, controls sound, saves or loads progress, and records scores on a leaderboard. The goal of the implementation is not only to create a playable game, but also to demonstrate how a medium-size program can be divided into cooperating objects with clear responsibilities.

The game uses a traditional update-and-draw loop. During each frame, the program reads input, updates the active game state, updates entities, checks collisions, and then draws the current state to the window. This structure makes OOP useful because many different objects need to react to the same actions, such as update and draw, while keeping their own specialized behavior.

Project Overview

The architecture follows a modular package structure. The central GAME package contains the main game loop, the window class, map loading, level switching, sound, persistence, and shared access through Handler. The Handler class is important because it gives entities and states controlled access to the current Game, Map, input managers, and GameFileManager without forcing every class to create those objects by itself.

The entity system represents everything interactive inside the game world. Entity is the abstract base type for all objects that can update, draw, and die. Creature extends Entity for moving objects, while StaticEntity is used for non-moving objects such as collectible dots. Concrete subclasses such as Player, RedGhost, BlueGhost, PinkGhost, OrangeGhost, and Dot implement the actual game behavior.

The state system separates the screens of the game. MenuState, GameState, AboutState, SoundState, and LeaderboardState all extend the abstract State class. This lets the program switch between menu, gameplay, level selection, sound options, and leaderboard without mixing all screen logic in one class.

The map is tile-based. Map reads a text file, converts numeric tile identifiers into Tile objects, stores them in a generic Grid<Tile>, and draws them on screen. Concrete tile classes such as Block, DotTile, gate tiles, corner tiles, and directional tiles extend Tile and customize whether the tile is solid and which image it uses.

Graphics are organized through Assets, ImageLoader, and SpriteSheet. Assets loads and exposes images, ImageLoader handles image loading and resizing, and SpriteSheet extracts smaller images from larger sprite sheets. Input is handled by KeyManager and Mouse, which implement Java listener interfaces. Persistence is handled by GameFileManager, which saves the current level, lives, score, and leaderboard values.

- GAME: game loop, window, map, levels, sound, file management, and shared Handler.
- GAME.Entities: abstract entity hierarchy and manager for active objects.
- GAME.Entities.MovingCreatures: Player and ghosts, all based on Creature.

- GAME.States: menu, gameplay, level selection, sound settings, and leaderboard screens.
- GAME.Tiles: reusable tile hierarchy for the maze.
- GAME.Graphics and GAME.Keys: image loading, sprite handling, keyboard input, and mouse input.

Inheritance

Inheritance is used when a class needs to reuse common attributes and behavior from a more general class. The project uses inheritance heavily in the entity, state, and tile hierarchies. For example, Player is a specialized Creature, and Creature is a specialized Entity.

```
public abstract class Entity {
    public abstract void Update() throws InvalidFileException;
    public abstract void Draw(Graphics g);
    public abstract void die();
}

public abstract class Creature extends Entity {
    public Creature(Handler handler, double x, double y) {
        this(handler, x, y, DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}

public class Player extends Creature implements Damageable {
    public Player(Handler handler, float x, float y) {
        super(handler, x, y);
    }
}
```

This hierarchy means Player does not need to reimplement the basic position, size, collision area, and movement structure already defined in Entity and Creature. The same idea is used by the ghost classes and by tile classes such as Block extending Tile.

Encapsulation

Encapsulation combines state and behavior inside a class. The internal fields are kept private, and other classes interact with them through methods. This protects the object from uncontrolled changes and keeps the rules for changing state inside the class that owns the data.

```
public abstract class Entity {
    private double x, y;
    private String direction = "right";
    private final Handler handler;
    private int width, height;
    private boolean active = true;

    public double getX() { return x; }
    public double getY() { return y; }
    public void setx(double x) { this.x = x; }
    public void sety(double y) { this.y = y; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
}
```

The position and active status of an Entity are not accessed directly from outside. Classes such as Player, Creature, and EntityManager use getters and setters, which makes the code easier to control and maintain.

Information Hiding

Information hiding separates the public interface from the private implementation. The public part shows what other classes can use, while the private part hides data and algorithms that should not be changed directly. This project demonstrates information hiding through private attributes and private helper methods.

```
public class GameWindow {
    private JFrame frame;
    private final String title;
    private final int width;
    private final int height;
    private Canvas canvas;

    public GameWindow(String title, int width, int height) {
        this.title = title;
        this.width = width;
        this.height = height;
        buildGameWindow();
    }

    private void buildGameWindow() {
        frame = new JFrame(title);
        canvas = new Canvas();
        frame.add(canvas);
    }

    public Canvas GetCanvas() { return canvas; }
    public JFrame getFrame() { return frame; }
}
```

The private attributes frame, title, width, height, and canvas are implementation details of the window. External classes do not construct or modify the window internals directly. They use public methods such as GetCanvas() and getFrame().

```
public abstract class Creature extends Entity {
    public void move() {
        moveX();
        moveY();
    }

    private void moveX() {
        // horizontal collision and movement algorithm
    }

    private void moveY() {
        // vertical collision and movement algorithm
    }
}
```

The public method move() is the visible interface. The algorithms moveX() and moveY() are hidden because other classes only need to ask the creature to move, not control every internal collision calculation.

Abstraction

Abstraction means defining a general concept and hiding the specific details behind a common interface. The project abstracts game screens with the State class. Every screen must update and draw, but each screen decides how those operations work.

```
public abstract class State {
    private static State currentState = null;
    private final Handler handler;

    public static void setState(State state) {
        State.currentState = state;
    }

    public static State getState() {
        return currentState;
    }

    public abstract void Update() throws InvalidFileException;
    public abstract void Draw(Graphics g);
}
```

MenuState, GameState, AboutState, SoundState, and LeaderboardState are different concrete states, but the Game class can work with them through the same State abstraction.

Polymorphism

Polymorphism allows objects of different concrete classes to be used through a shared type. This project uses inclusion polymorphism, method overloading, parametric polymorphism, and type coercion or casting.

- Polymorphism by Inclusion

Polymorphism by inclusion appears when subclasses are treated as objects of a parent type. EntityManager stores many different objects in one ArrayList<Entity> and calls the same methods on each object.

```
public class EntityManager {
    private ArrayList<Entity> entities;

    public void Update() throws InvalidFileException {
        Iterator<Entity> it = entities.iterator();
        while (it.hasNext()) {
            Entity e = it.next();
            e.Update();
            if (!e.isActive()) {
                it.remove();
            }
        }
    }

    public void Draw(Graphics g) {
        for (Entity e : entities) {
            e.Draw(g);
        }
    }
}
```

At runtime, Java dispatches to the correct Update() or Draw() implementation for Player, Dot, or a ghost. The manager does not need separate loops for every concrete class.

- Method Overloading

Method overloading allows multiple methods or constructors to share a name while using different parameter lists. Creature has two constructors: one with custom dimensions and one with default dimensions.

```
public abstract class Creature extends Entity {
    public Creature(Handler handler, double x, double y,
                    int width, int height) {
        super(handler, x, y, width, height);
    }

    public Creature(Handler handler, double x, double y) {
        this(handler, x, y, DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}
```

ImageLoader also overloads LoadImage so an image can be loaded normally or loaded and resized in one call.

```
public static BufferedImage LoadImage(String path)
        throws ImageNotFoundException {
    return ImageIO.read(ImageLoader.class.getResource(path));
}

public static BufferedImage LoadImage(String path, int width, int height)
        throws ImageNotFoundException {
    BufferedImage originalImage = LoadImage(path);
    BufferedImage resizedImage =
        new BufferedImage(width, height, originalImage.getType());
    return resizedImage;
}
```

- Parametric Polymorphism

Parametric polymorphism is implemented with Java generics. Grid<T> can store any type T while still keeping type safety.

```
public class Grid<T> {
    private final List<List<T>> grid;

    public void setCell(int row, int col, T item) {
        if (isValid(row, col)) {
            grid.get(row).set(col, item);
        }
    }

    public T getCell(int row, int col) {
        if (isValid(row, col)) {
            return grid.get(row).get(col);
        }
        return null;
    }
}
```

```
private Grid<Tile> gameTiles;
gameTiles = new Grid<>(height, width);
```

The Map class uses Grid<Tile>, so the grid is reusable but still restricted to Tile objects in this context.

- Polymorphism by Coercion and Type Casting

Coercion and casting allow a value to be treated as another compatible type. The project uses numeric casts for drawing and runtime type checks for collision behavior.

```
public void Draw(Graphics g) {
    BufferedImage image = directionImages.get(getDirection());
    g.drawImage(image, (int) getX(), (int) getY(), null);
}
```

```
for (Entity e : getHandler().getMap()
        .getEntityManager().getEntities()) {
    if (e instanceof Dot) {
        e.setActive(false);
        getHandler().getGame().setScore(10);
    }
}
```

The cast from double to int is needed because drawing coordinates must be integers. The instanceof check lets the program apply dot-specific logic only when the collided Entity is actually a Dot.

Interface Implementation

An interface defines a contract that a class must follow. The project defines Damageable for objects that can receive damage.

```
public interface Damageable {
    void takeDamage(int amount);
}

public class Player extends Creature implements Damageable {
    @Override
    public void takeDamage(int amount) {
        System.out.println("Player taking " + amount + " damage!");
        die();
    }
}
```

The project also uses Java event interfaces for input handling.

```
public class KeyManager implements KeyListener {
    @Override
    public void keyPressed(KeyEvent e) {
        keys[e.getKeyCode()] = true;
    }
}

public class Mouse implements MouseListener, MouseMotionListener {
    @Override
    public void mouseMoved(MouseEvent e) {
        x = e.getX();
        y = e.getY();
    }
}
```

Composition

Composition is a has-a relationship. A complex object is built from smaller objects that each have their own responsibility. The Game class is composed of a window, keyboard manager, mouse manager, state references, and a file manager.

```
public class Game implements Runnable {
    private GameWindow wnd;
    private final KeyManager key;
    private final Mouse mouse;
    private State menuState;
    private final GameFileManager gameFileManager;

    public Game(String title, int width, int intHeight,
                GameFileManager gameFileManager) {
        key = new KeyManager();
        mouse = new Mouse();
        this.gameFileManager = gameFileManager;
    }
}
```

The Map class is also composed of a grid and an entity manager.

```
public class Map {
    private Grid<Tile> gameTiles;
    private final EntityManager entityManager;

    public Map(Handler handler, String path)
            throws InvalidFileException {
        entityManager =
            new EntityManager(handler, new Player(handler, 100, 100));
        loadMap(path);
    }
}
```

Subtyping

Subtyping means a derived type can be used where a base type or interface is expected. The developer side of subtyping is visible in the class declarations, where the relationships are created.

```
public class Player extends Creature implements Damageable {
    // Player is a Creature, an Entity, and a Damageable object.
}

public class RedGhost extends Creature {
    // RedGhost is a Creature and an Entity.
}

public class MenuState extends State {
    // MenuState is a State.
}

public class Block extends Tile {
    // Block is a Tile. }
```

These declarations are the developer-side proof of subtyping. They create the type relationships that later allow polymorphic code such as ArrayList<Entity> and State.setState(new GameState(...)).

Exception Handling

Exception handling lets the program react to invalid files, missing resources, malformed data, and other runtime problems without mixing error logic into normal game logic. The project uses Java built-in exceptions and custom exceptions.

```
public class InvalidFileException extends Exception {
    InvalidFileException(String message) {
        super(message);
    }
}

public class ImageNotFoundException extends Exception {
    ImageNotFoundException(String message) {
        super(message);
    }
}
```

```
public static String loadFile(String path)
        throws InvalidFileException {
    StringBuilder builder = new StringBuilder();
    try {
        BufferedReader b = new BufferedReader(new FileReader(path));
        String line;
        while ((line = b.readLine()) != null) {
            builder.append(line).append("\n");
        }
        b.close();
    } catch (Exception e) {
        throw new InvalidFileException("The file was not found!");
    }
    return builder.toString();
}
```

GameFileManager also catches IOException, NumberFormatException, and NullPointerException when loading saved progress. If the save file is missing or invalid, it falls back to a new game state.

Customization

Customization in this project means behavior that is not fixed to one hard-coded path and can change based on user choice or saved user data. The current project does not implement choosing a different player character, but it does support level selection, sound choice, save/load progress, and leaderboard persistence.

The player can choose the level from AboutState. The selected level changes the active GameState and saved game data.

```
if (getHandler().getMouse().pressed_left()
        && level4.contains(getHandler().getMouse().getX(),
                           getHandler().getMouse().getY())) {
    getHandler().getGame().setLevel(4);
    getHandler().getGameFileManager().saveGame(4, 5, 0);
    State.setState(new GameState(getHandler(), 4));
}
```

The player can load previous progress from the main menu.

```
if (getHandler().getMouse().pressed_left()
        && load.contains(getHandler().getMouse().getX(),
                         getHandler().getMouse().getY())) {
    int[] savedData = getHandler().getGameFileManager().loadGame();
    getHandler().getGame().setLevel(savedData[0]);
    getHandler().getGame().setScoreValue(savedData[2]);
    State.setState(new GameState(getHandler(), savedData[0]));
}
```

The sound screen lets the user turn off the game sounds.

```
if (getHandler().getMouse().pressed_left()
        && yes.contains(getHandler().getMouse().getX(),
                        getHandler().getMouse().getY())) {
    Sound.getBeginningSound().setSoundOff(1);
    Sound.getChompSound().setSoundOff(1);
    Sound.getDeathSound().setSoundOff(1);
    State.setState(getHandler().getGame().getMenuState());
}
```

Save/load and leaderboard files also personalize the game because the result depends on the user's previous progress and scores instead of a single fixed initial state.

Open/Closed Principle

The Open/Closed Principle states that classes should be open for extension but closed for modification. The project supports this idea through abstract base classes and polymorphic loops. New states, entities, and tiles can be added by extending the existing base types.

```
public class Block extends Tile {
    public Block(int idd) {
        super(Assets.getBlock(), idd);
    }

    @Override
    public boolean IsSolid() {
        return true;
    }
}
```

```
for (Entity e : entities) {
    e.Draw(g);
}

if (State.getState() != null) {
    State.getState().Update();
}
```

The loops work with Entity and State references, so adding a new enemy or screen does not require rewriting the main draw/update mechanism. The tile hierarchy works similarly: each new tile can override behavior such as IsSolid() while the map still works with the general Tile type.